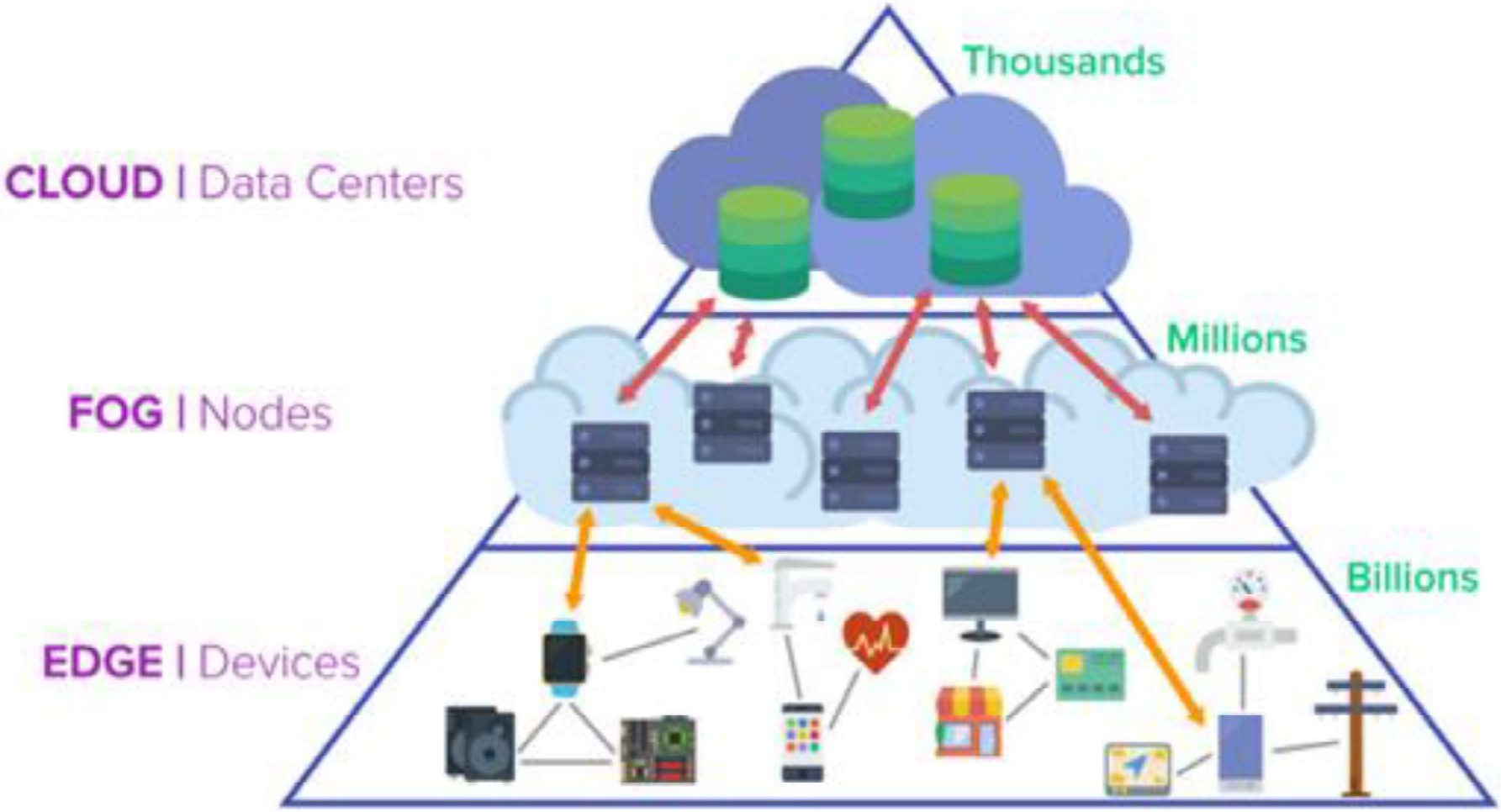


Développement des Applications Réparties

Dr ZAKARIA BENZADRI

Zakaria.benzadri@univ-constantine2.dz

Nouvelle Génération des Applications Réparties



Introduction

Les applications réparties connaissent une croissance exponentielle aussi bien en termes de taille que de complexité.

A ce titre, la conception et le développement des applications réparties ont connu un intérêt croissant pendant la dernière décennie, et sont ainsi devenues un sous-domaine central du Génie Logiciel.

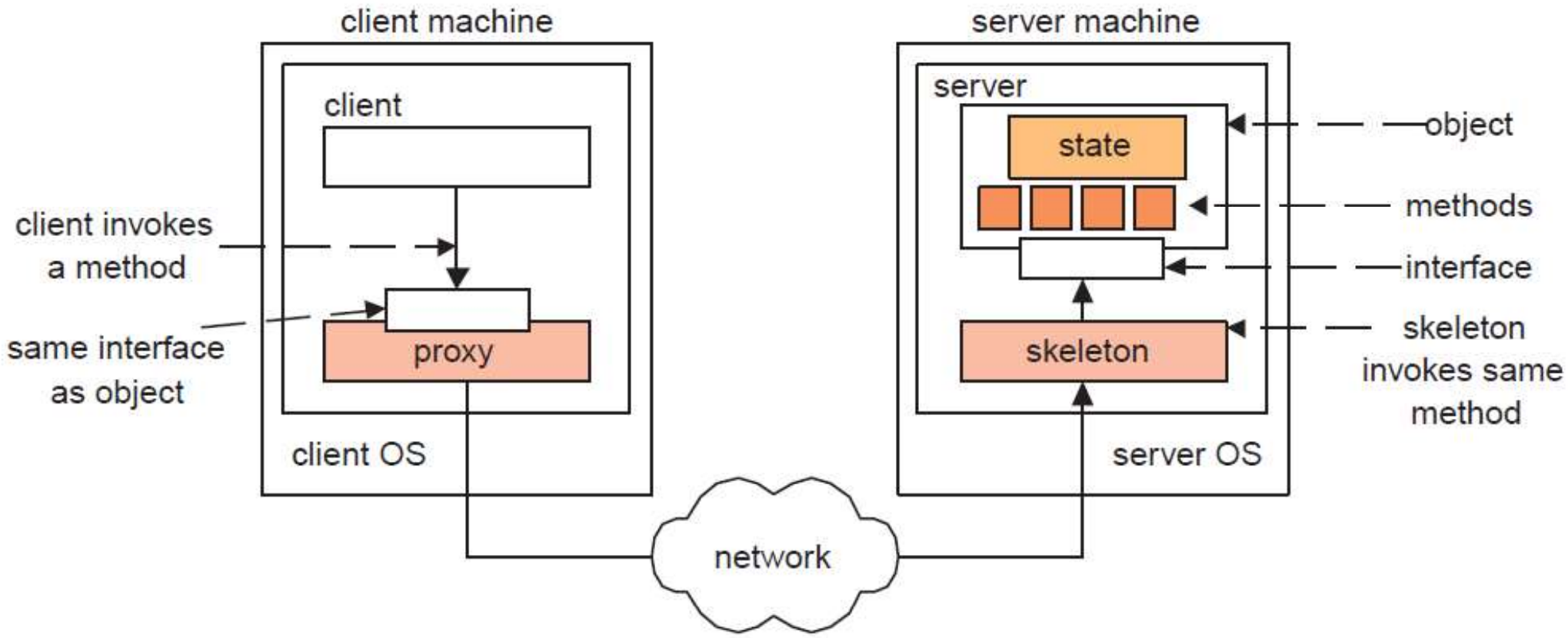
Pour concevoir ces applications de plus en plus complexes, la Programmation Orientée Composant (POC) a gagné en pouvoir d'expression et en abstraction.

En effet, en offrant une vue architecturale modulaire des composants et leurs dépendances, cette représentation permet de gérer la croissance fulgurante des applications réparties.

Introduction

- Programmation procédurale → (-) effets de bord
- Programmation par objets → (+) modularité (-) réutilisation et évolution.
- **Programmation orientée composant** → assemblage de composants prédéfinis, chacun d'entre eux joue un rôle spécifique dans le système et définit clairement les services qu'il offre et les services qu'il requiert, sous forme d'interfaces, pour accomplir sa fonctionnalité.

POO	POC
les objets encapsulent les données et le comportement,	Les composants sont indépendants et déployables,
est basée objets,	est basée interfaces,
est une technologie d'implémentation,	est une technologie de distribution et d'empaquetage,
supporte la réutilisation de bas niveau,	supporte la réutilisation de haut niveau,
Utilise les langages orientés objets,	utilise n'importe quel langage,
possède des formes de connexion assez limitées (invocation de méthodes).	possède des mécanismes adéquats pour la composition(délégation, aggrégation).


client machine
client
client invokes
a method
same interface
as object
proxy
client OS
server machine
server
state
object
methods
interface
skeleton
skeleton
invokes same
method
server OS
network

Cycle de développement de composants

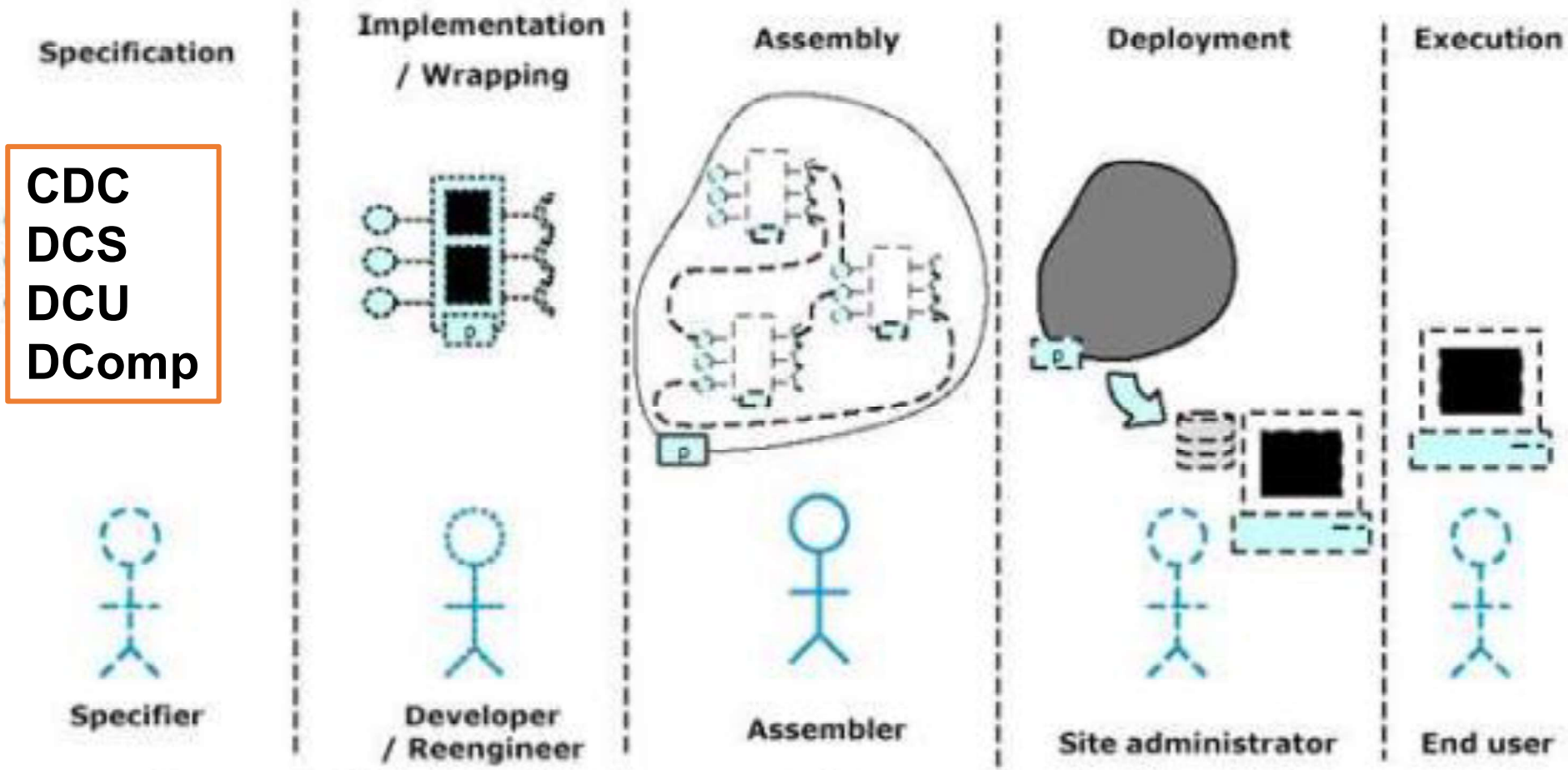
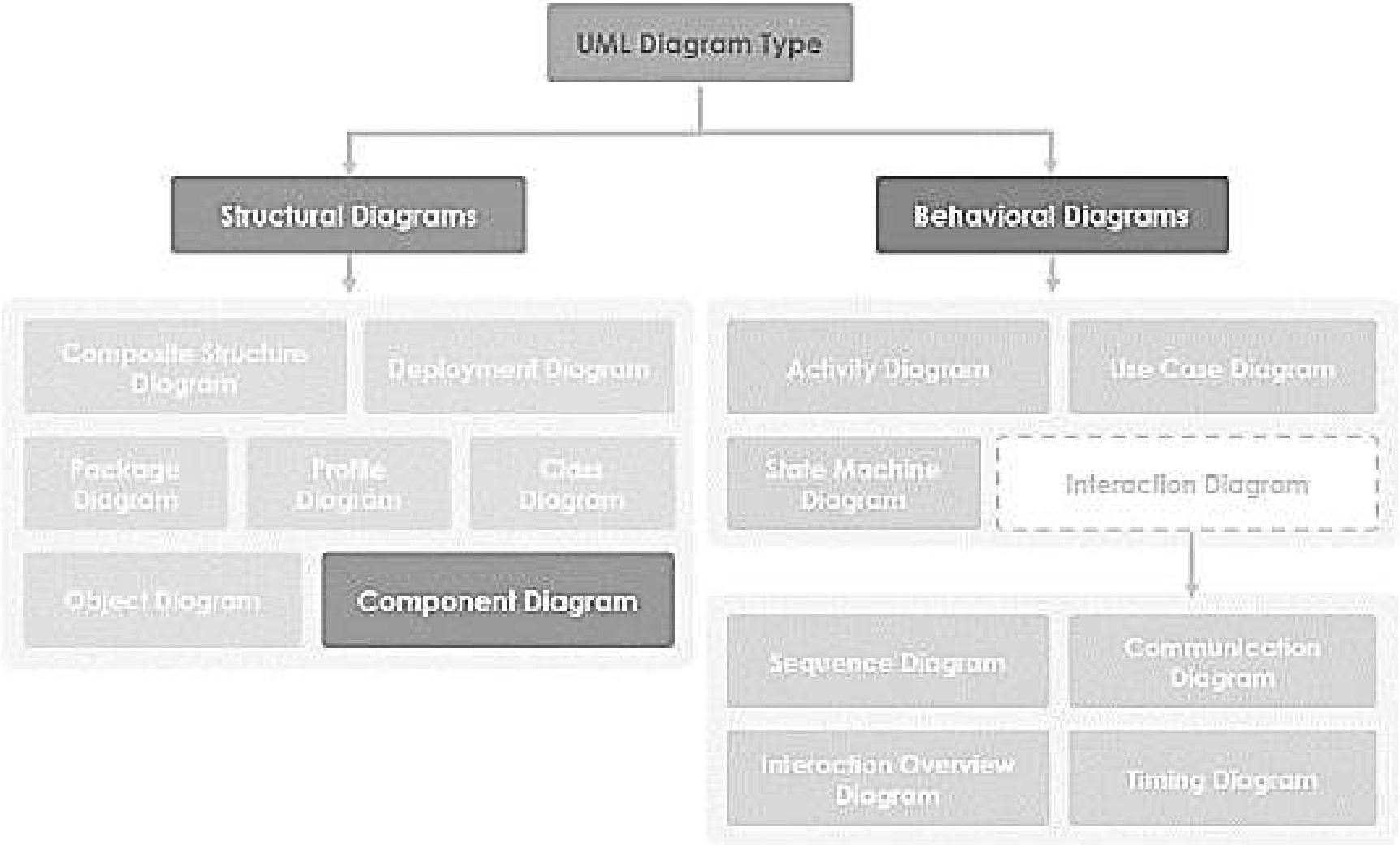


Figure 3. Cycle de développement d'une application à base de composants.

Principaux diagrammes UML



Composant

- **Un composant a une collection de propriétés, de méthodes et d'événements.**
  - Les Propriétés représentent les états ou les attribues d'un composant. Chaque propriété a un type.
  - Les Méthodes décrivent le comportement et **les services** d'un composant. Chaque méthode a une signature.
  - Les Événements décrivent les actions à entreprendre par un composant. Chaque événement possède aussi un type.

- **Les interfaces qui sont un sous-ensemble du produit cartésien des ensembles propriétés, méthodes et événements. Elles définissent les parties de communication d'un composant avec le monde extérieur.**

Notion de composant et d'interface:

- **En UML, un composant est un élément logiciel remplaçable et réutilisable qui fournit ou reçoit un service bien précis.**
- **Il peut être vu comme une pièce détachée du logiciel.**

- **La notion de composant est proche de celle d'objet, dans le sens de la modularité et de réutilisation.**

- **Le composant est l'architecture du logiciel ce que l'objet est à l'architecture du code.**

Notion de composant et d’interface:

- **Les composants fournissent des services via des interfaces.**
- **Un composant peut être remplacé par n’importe quel autre composant compatible c'est-à-dire ayant les mêmes interfaces.**
- **Un composant peut évoluer indépendamment des applications ou des autres composants qui l’utilise à partir du moment ou les interfaces sont respectées.**

Types d'interfaces

- **Les interfaces requises :**
  - Ce sont des interfaces qui fournissent un service au composant et dont il a besoin pour fonctionner.

- **Les interfaces fournies :**
  - Ce sont des interfaces par lesquels le composant fournit lui-même un service.

Composant UML

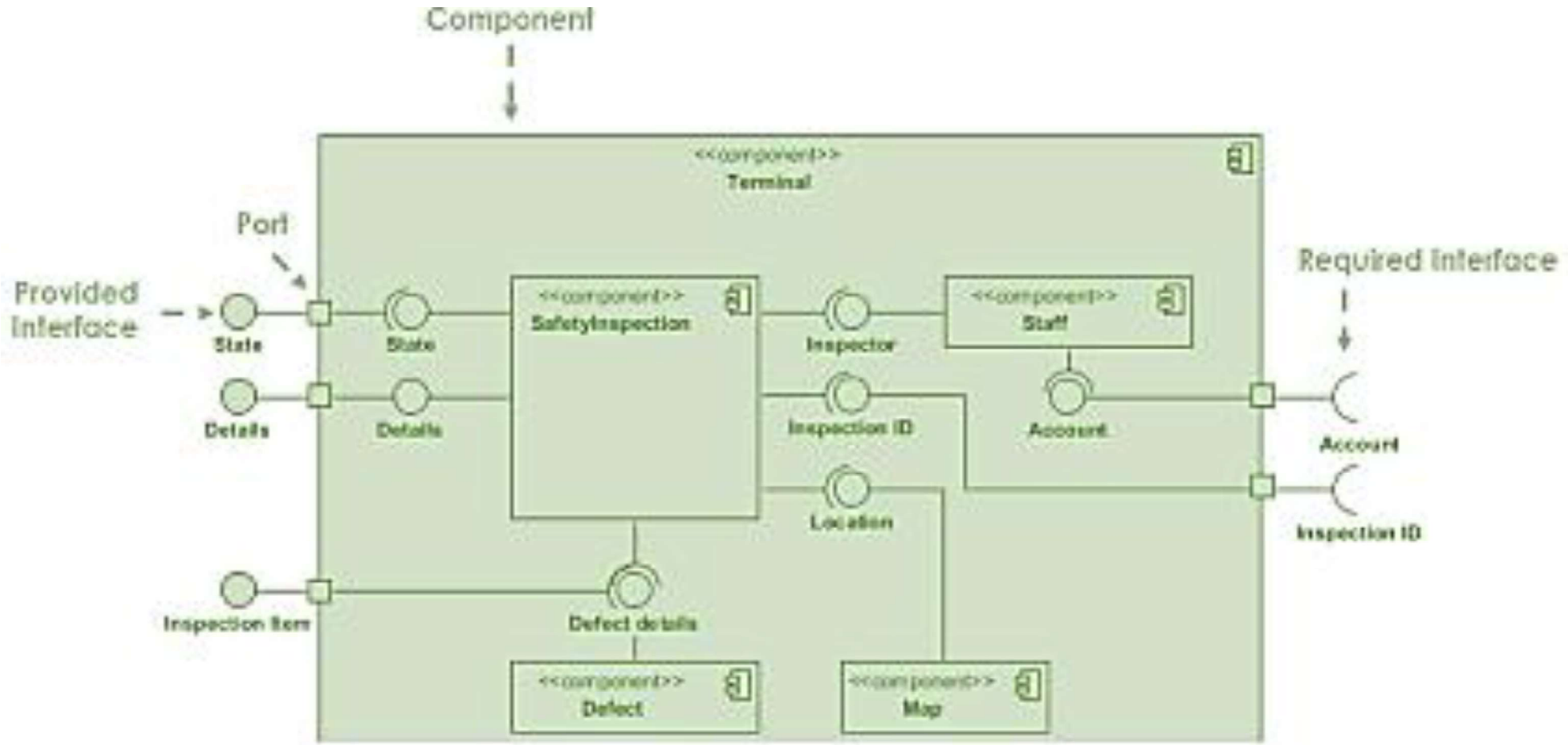
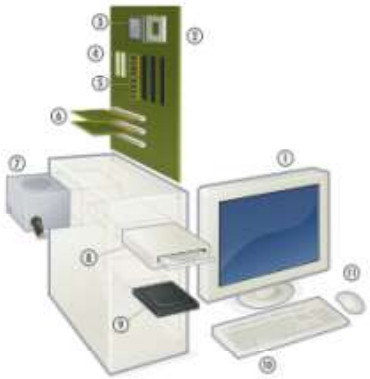


Figure 10. Structure interne d'un composant

Exemple



Ex : parallèle avec les composants d'un ordinateur.

Un ordinateur est un ensemble de composants modulaires qui fournissent et reçoivent des services (carte mère, carte graphique, disque dur, clavier, écran...). Chacun de ces composants est remplaçable par un autre composant (pas forcement identique) à condition qu'il ait des interfaces compatibles (nous ne pouvons pas mettre un écran avec une connexion VGA à la place d'un écran avec une connexion HDMI).

ATTENTION : ceci n'est qu'un parallèle pour faire comprendre la notion de composant. ***En UML les composants ne sont pas des éléments matériels mais des éléments logiciels.*** Éléments logiciels qui par contre seront installés sur des éléments matériels (ce que nous verrons lorsque nous aborderons le diagramme de déploiement).

Représentation graphique : composant



Un rectangle dans lequel figure :

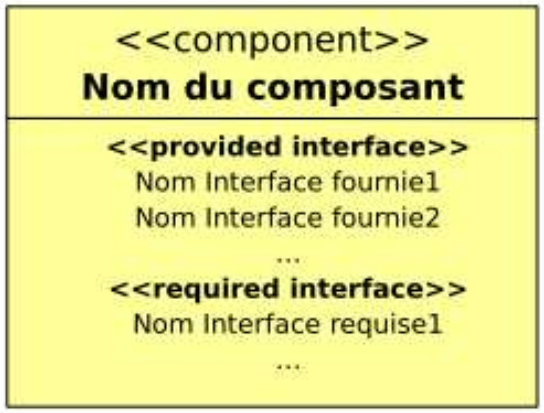
- Le nom du composant.
- Le symbole  en haut à droite.



Un rectangle dans lequel figure :

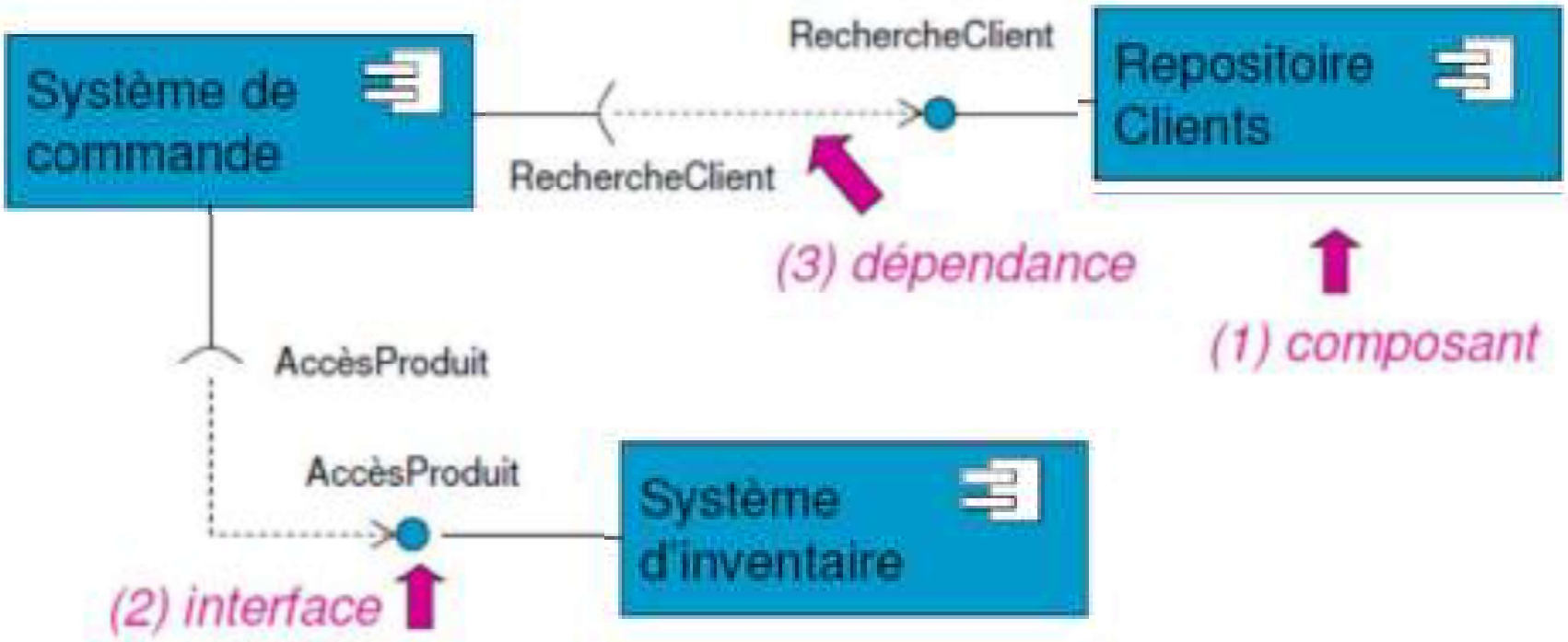
- Le stéréotype **<<component>>**.
- Le nom du composant.

Représentation graphique : interfaces



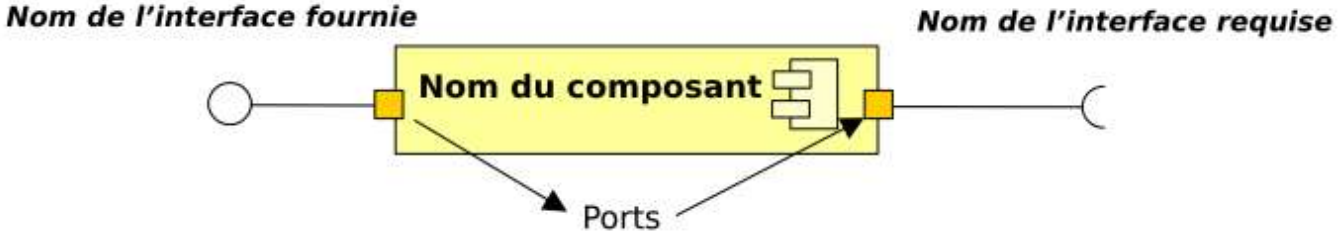


Exemple


Système de commande
RechercheClient
RechercheClient
Repositoire Clients
(3) dépendance
(1) composant
AccèsProduit
AccèsProduit
Système d'inventaire
(2) interface

Représentation graphique : port

Le port est le point de connexion entre le composant et son environnement, il est la matérialisation de l'interface. Nous le représentons par un petit carré à la périphérie du composant.

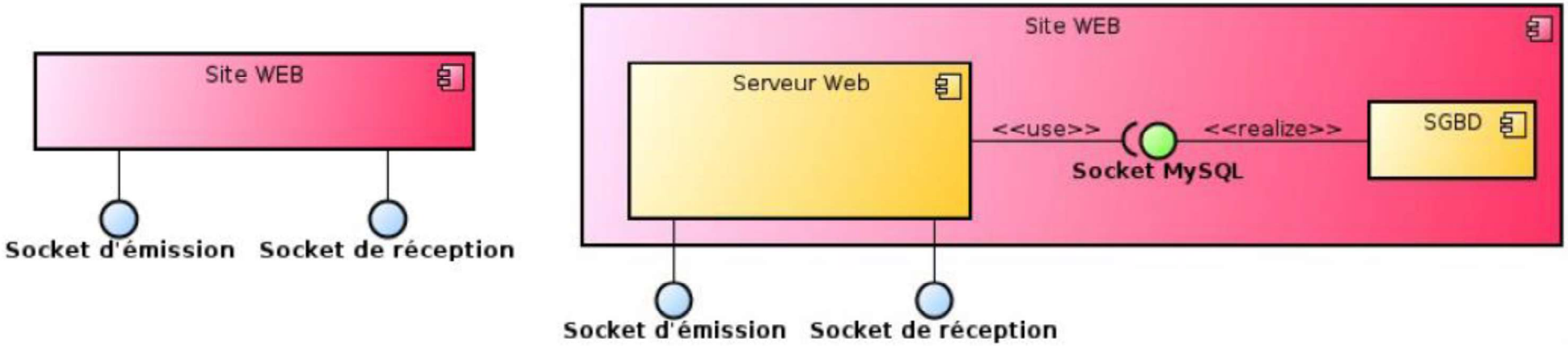


Composant composite

Un composant peut être vu de 2 manières :

- ✗ Comme une boite noire dont nous ne connaissons pas le contenu et auquel nous accédons via les interfaces qui sont la seule partie visible.
- ✗ Comme une boite blanche en spécifiant les objets qui constituent le composant et en indiquant leurs relations.

Ex : Transfert de données par Internet vu précédemment.



Rôle du diagramme de composants :

- **Le diagramme de composants fait parti des diagrammes structuraux (statiques) d'UML. Il permet de représenter les différents éléments logiciels (composants) du système et leurs dépendances (relations qui les lient).**

- **Les dépendances de composants peuvent être :**
  - Des relations de compositions
  - Des relations d'assemblages et de connexion (via des interfaces)

- **Élaborer le diagramme de composants d'une application web.**

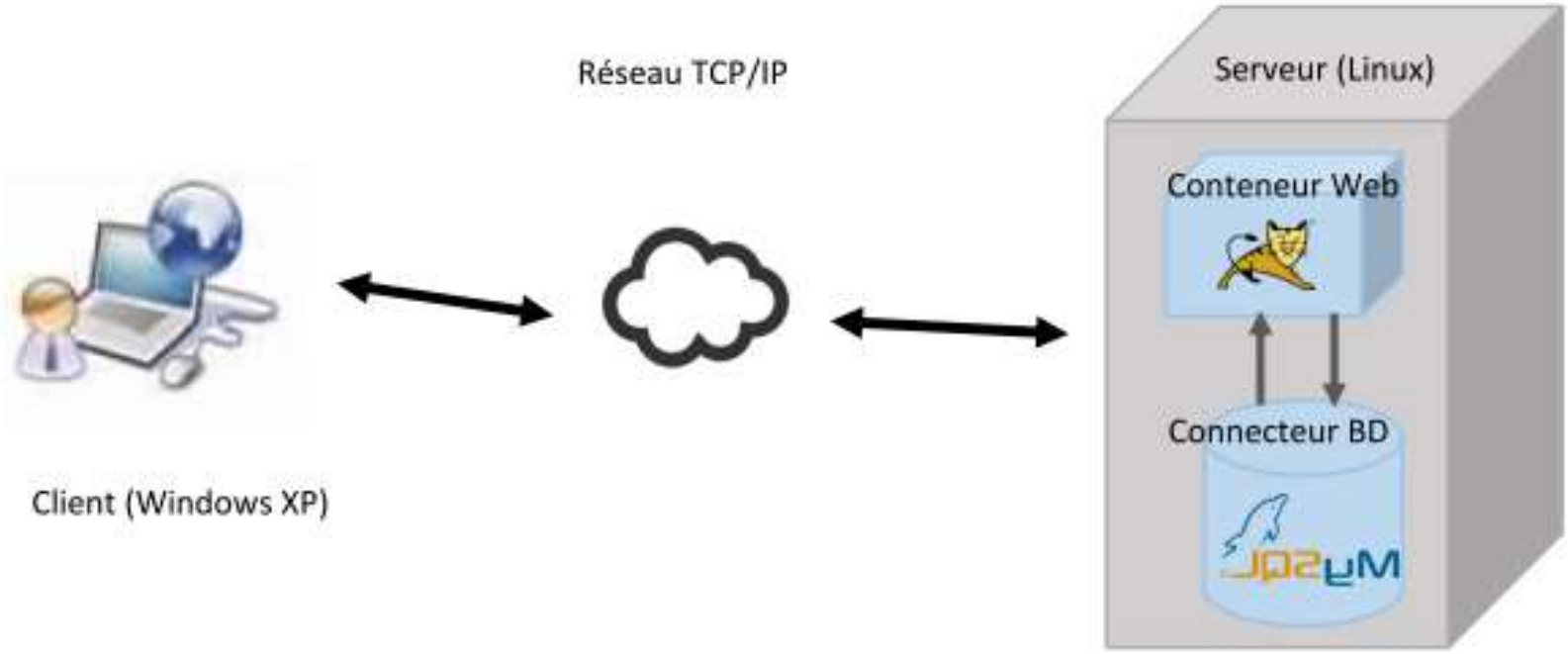
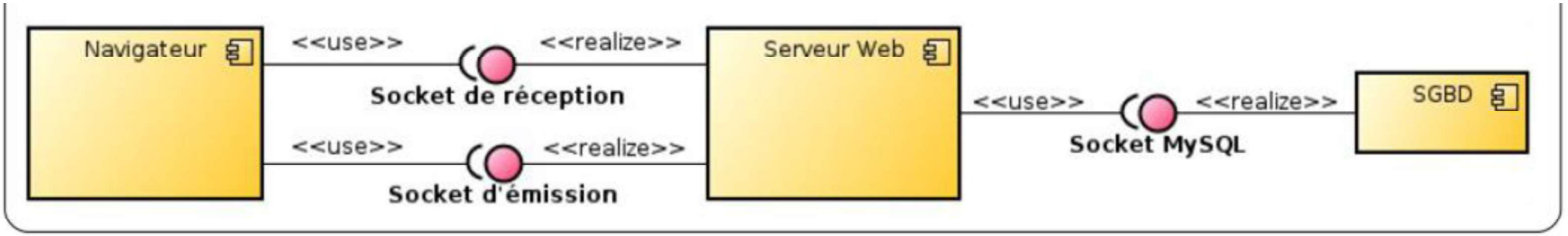
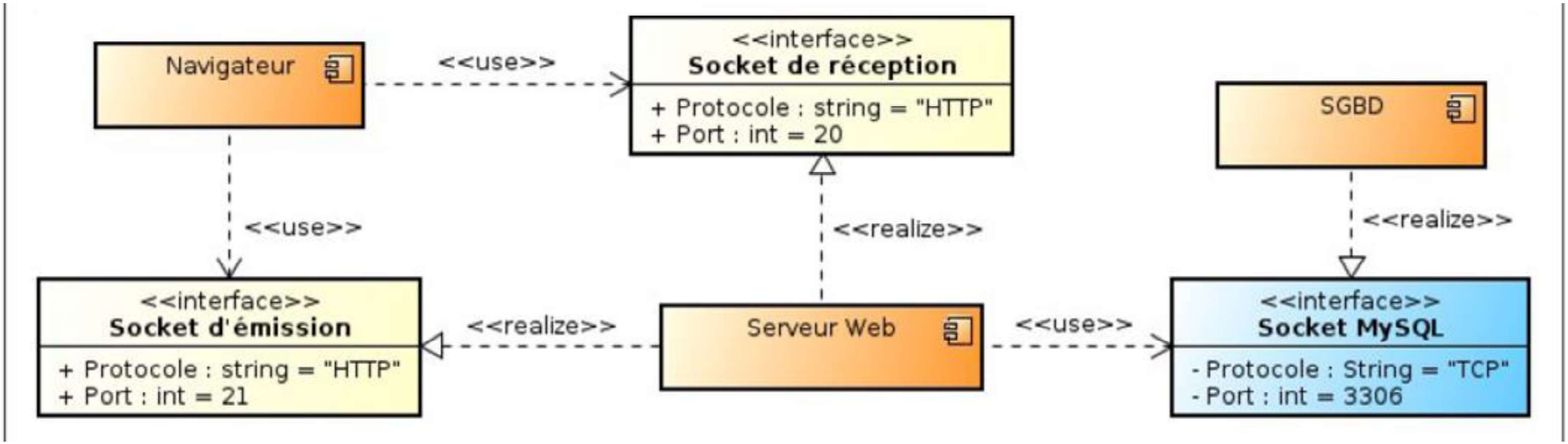


Figure : Architecture Client/ Serveur

TD 1 Corr



Ou bien

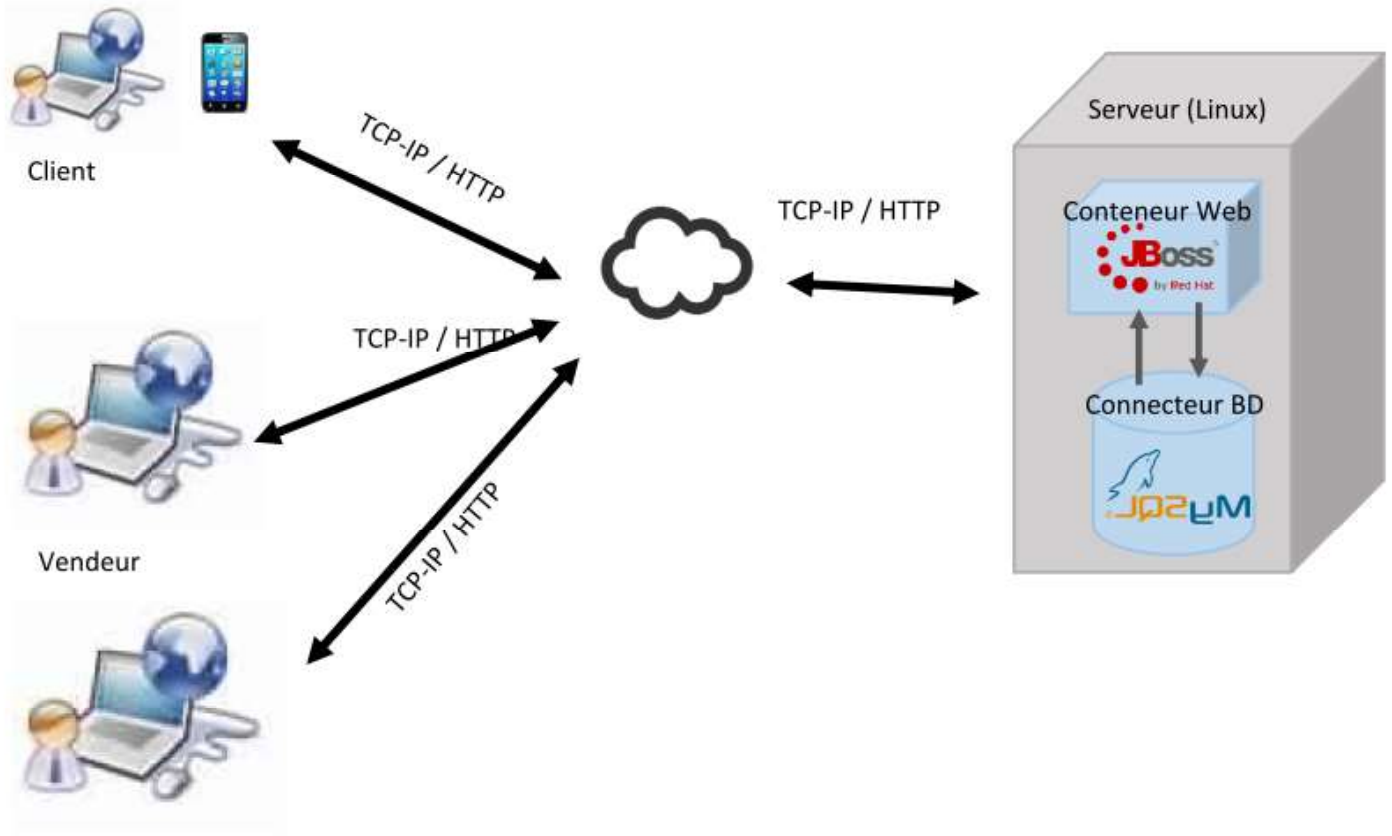


TD2

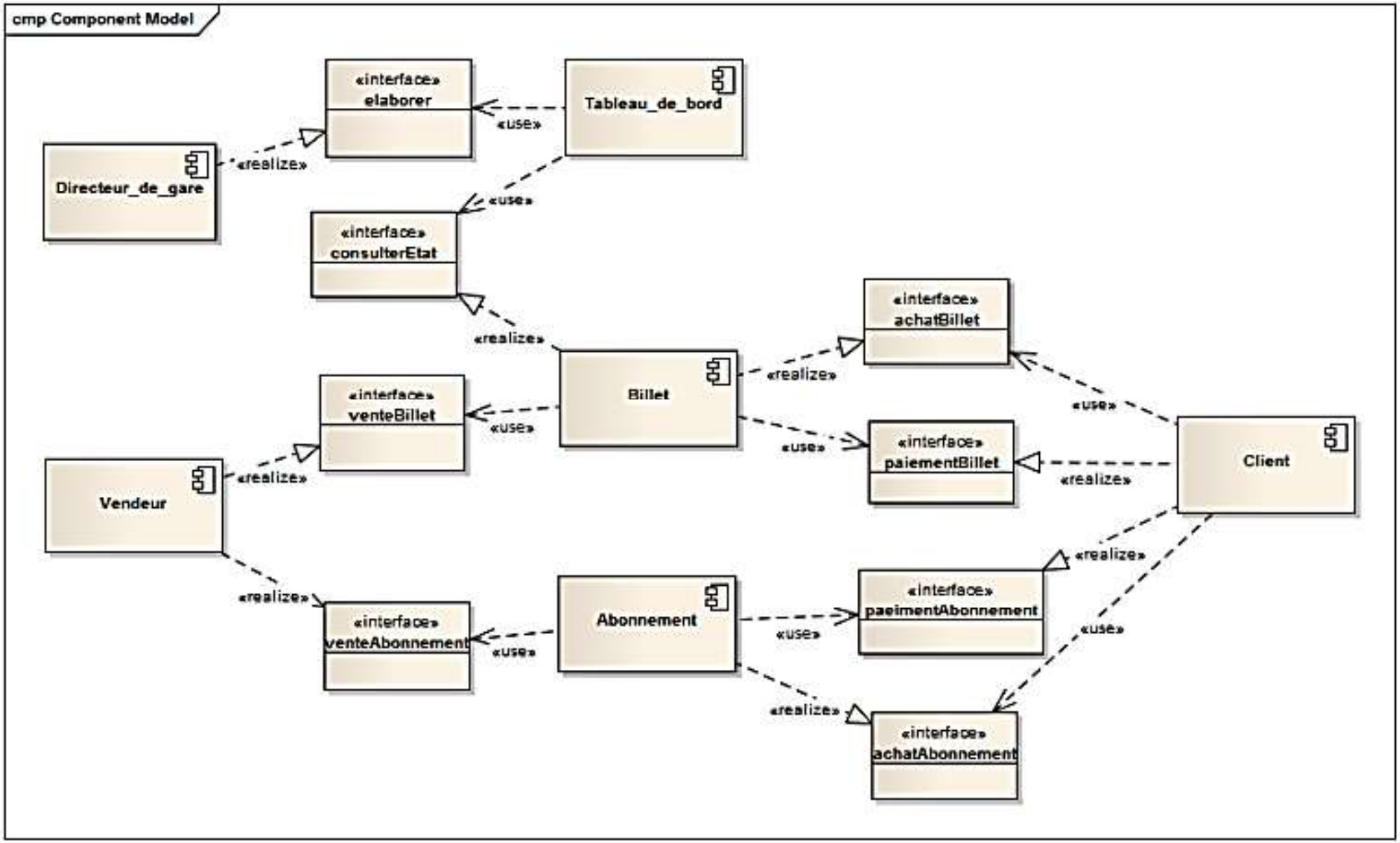
- **La Société Nationale des Chemins de Fer souhaite concevoir et développer une application de vente des billets et des abonnements. Cette application permet:**
  - à ses clients d'acheter des billets ou des abonnements
  - à ses vendeurs aux guichets d'effectuer des ventes groupées des billets ou des abonnements.
  - aux directeurs des gares de consulter les états des billets et d'élaborer des tableaux de bord.
  - Le paiement des billets et des abonnements s'effectue par carte bancaire.
  - L'application doit offrir des interfaces personnalisées aux clients, aux vendeurs de guichets et aux directeurs des gares
- **1 Proposer une architecture technique pour cette application**
- **2 Quels sont les composants principaux de**
- **cette application ?**
- **3 Élaborer le diagramme de composants**



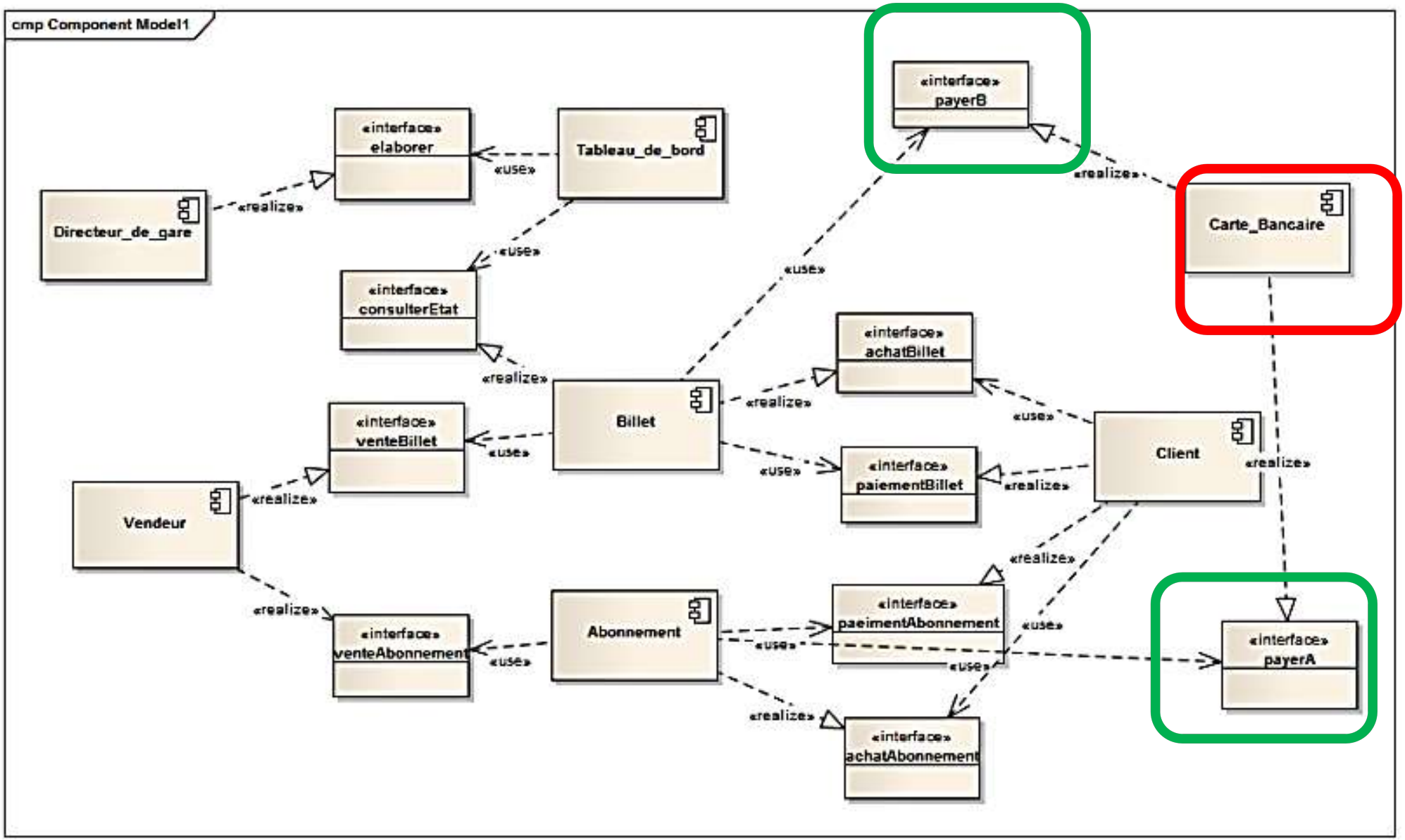
TD 2 Corr


Client
TCP-IP / HTTP
TCP-IP / HTTP
Serveur (Linux)
Conteneur Web
JBoss
by Red Hat
Connecteur BD
MySQL
TCP-IP / HTTP
Vendeur
TCP-IP / HTTP

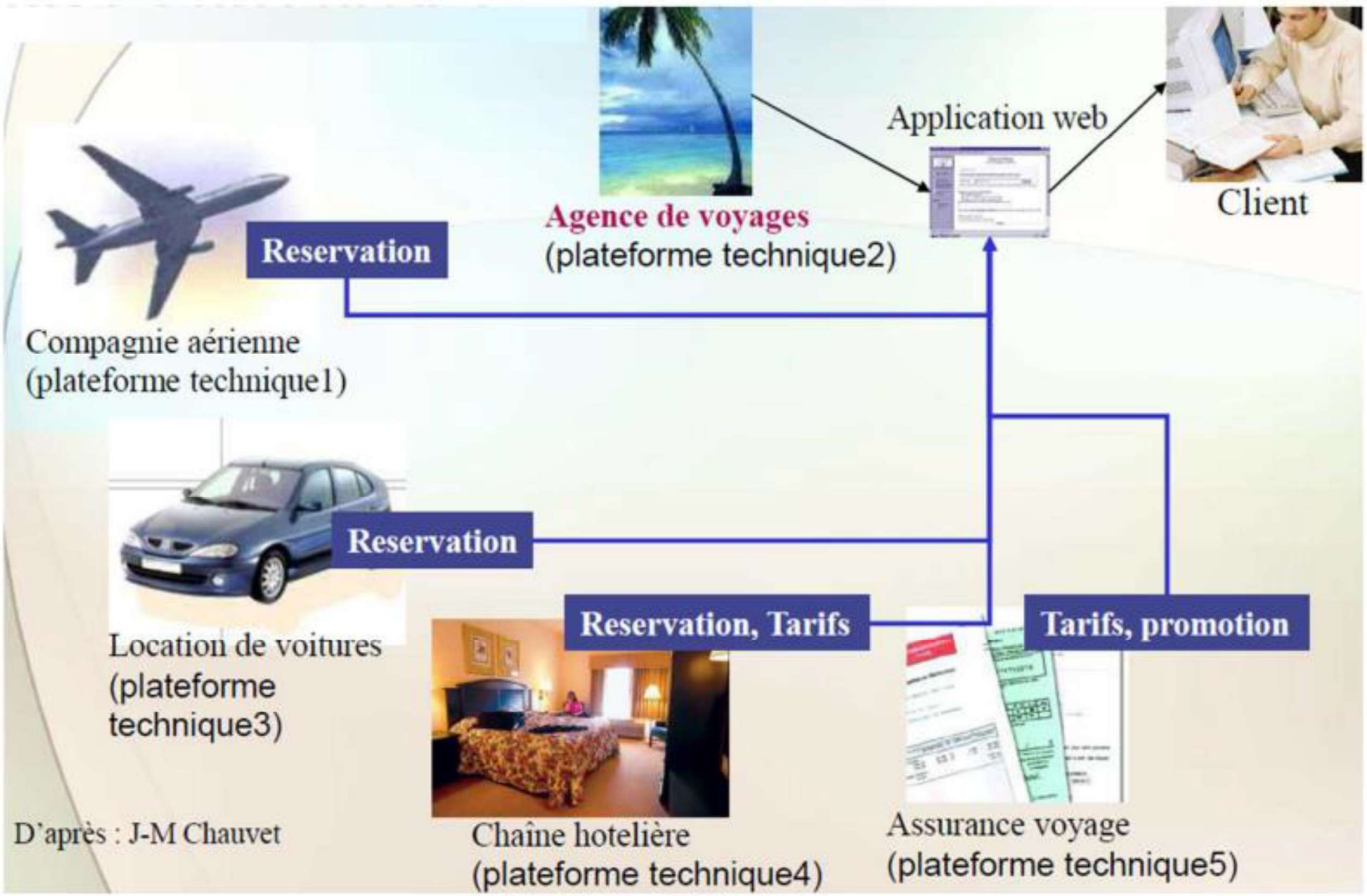
TD 2 Corr


cmp Component Model
«interface»
elaborer
Tableau_de_bord
«use»
Directeur_de_gare
«realize»
«use»
«interface»
consulterEtat
«realize»
«interface»
achatBillet
Billet
«realize»
«interface»
venteBillet
«use»
«use»
«interface»
paiementBillet
Client
Vendeur
«realize»
«realize»
«realize»
«interface»
venteAbonnement
Abonnement
«use»
«interface»
paeimentAbonnement
«use»
«use»
«realize»
«interface»
achatAbonnement

TD 2 V2


cmp Component Model1
«interface»
elaborer
Tableau_de_bord
«use»
Directeur_de_gare
«realize»
«use»
«interface»
consulterEtat
«realize»
«interface»
payerB
«realize»
Carte_Bancaire
«use»
«interface»
achatBillet
«realize»
Billet
«interface»
venteBillet
«use»
«use»
«interface»
paiementBillet
«realize»
Client
«use»
«realize»
Vendeur
«realize»
«realize»
«realize»
«interface»
paeimentAbonnement
«use»
«use»
«use»
«interface»
payerA
«interface»
venteAbonnement
«use»
Abonnement
«realize»
«interface»
achatAbonnement

TD 3



1 Proposer une architecture technique pour cette application
2 Quels sont les composants principaux de cette application ?
3 Élaborer le diagramme de composants

Les avantages à utiliser une approche POC

- **spécialisation:** L'équipe de développement peut-être divisée en sous-groupes, chacun se spécialisant dans le développement d'un composant,
- **sous traitance:** Le développement d'un composant peut-être externalisé, à condition d'en avoir bien réalisé les spécifications au préalable,
- **facilité de mise à jour:** La modification d'un composant ne nécessite pas la recompilation du projet complet,
- **facilité de livraison/déploiement:** Dans le cas d'une mise à jour, d'un correctif de sécurité, etc., alors que le logiciel à déjà été livré au client, la livraison en est facilitée, puisqu'il n'y a pas besoin de re-livrer l'intégralité du projet, mais seulement le composant modifié,
- **choix des langages de développement:** Il est possible, dans la plupart des cas, de développer les différents composants du logiciel dans des langages de programmation différents. Ainsi, un composant nécessitant une fonctionnalité particulière pourra profiter de la puissance d'un langage dans un domaine particulier, sans que cela n'influe sur le développement de l'ensemble du projet,
- **productivité:** La réutilisabilité d'un composant permet un gain de productivité non négligeable car elle diminue le temps de développement, d'autant plus que le composant est réutilisé souvent.